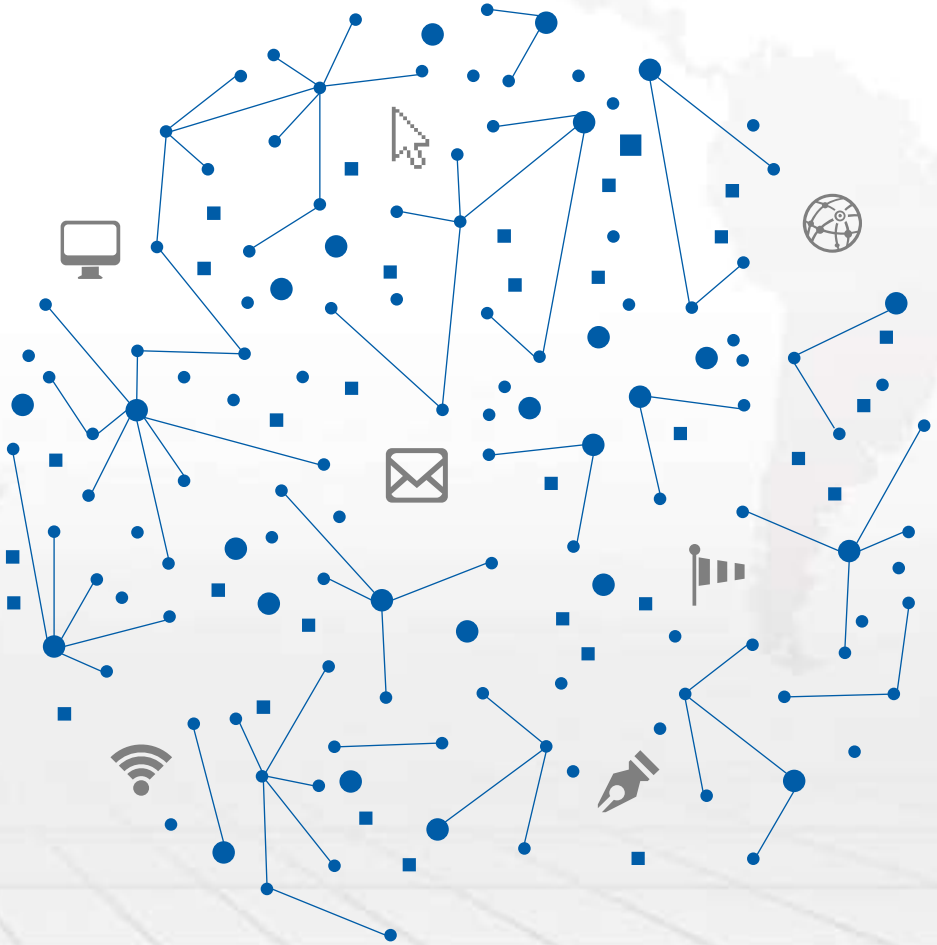
# Autonomous Mobile Robot for Maze Navigation and Mapping with 2D LiDAR

## Design & Build Training Project

**Group 28**



# CONTENT


A Project Introduction

B Team Roles

C Module Introduction

D Completion

# Project Introduction

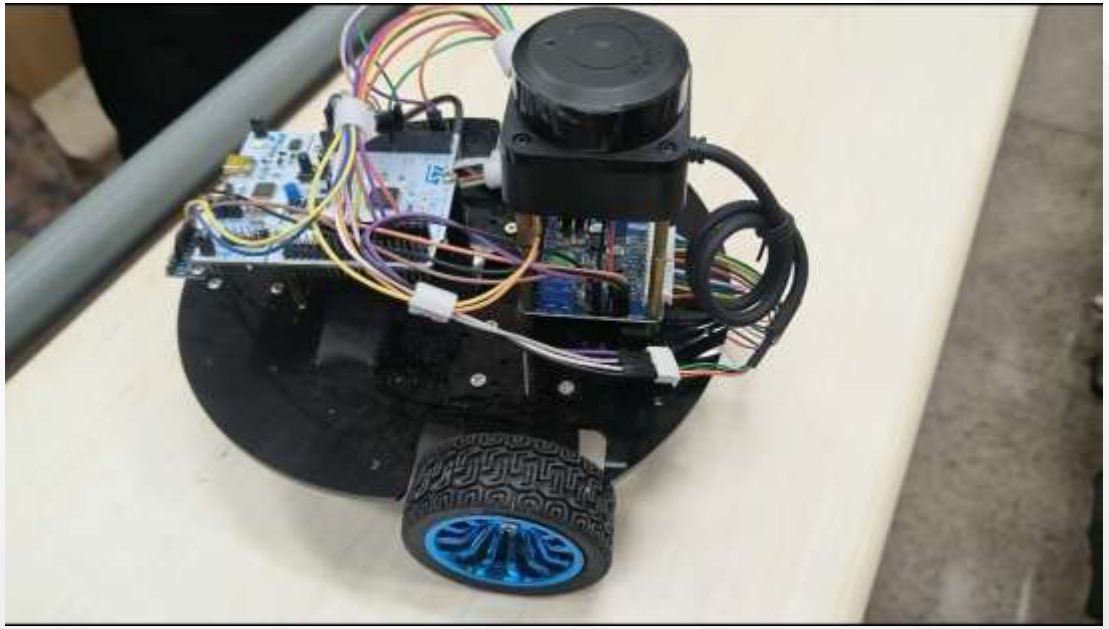
# Car system

**Core controller:** STM32 microcontroller

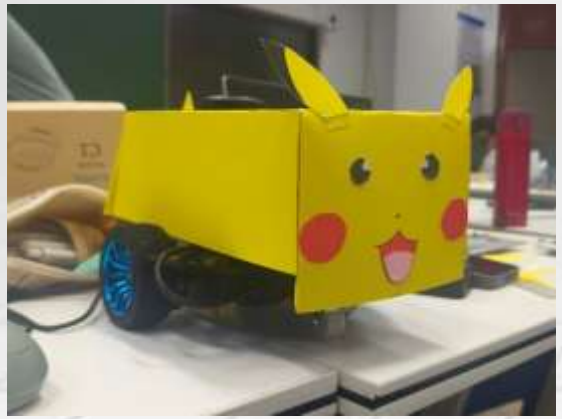
**Sensing devices:** 2D LiDAR + MPU

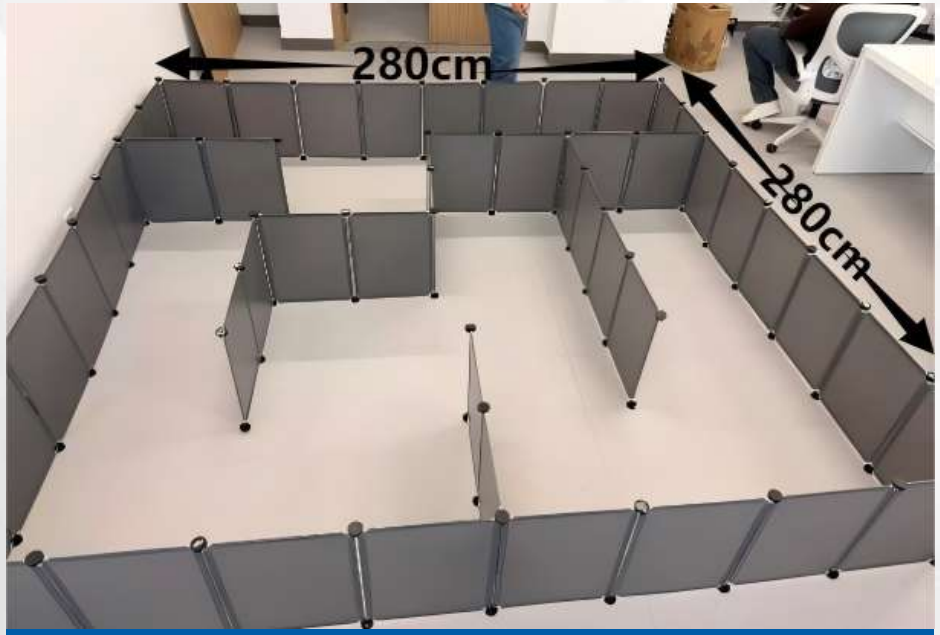
**Software algorithms:** SLAM real-time mapping + path planning (A*) + BFS

**Communication method:** Bluetooth wireless transmission



Physical appearancece of the robot (video)






Physical map of the maze

## Map environment

**Maze structure:** 4×4 grid, 280cm×280cm

**Unit size:** 70cm×70cm

**Key feature:** Fixed layout but random entrance and exit (to be announced before acceptance)

**Navigation target:** From entrance to exit, then return to the entrance

# Team Roles

# Task division (Hardware)

| Group members | Task division |
|---|---|
| Jinglin Wang (leader) | 1.Led the code development for **the MPU6500 module.**<br>2. Connected and debugged the Bluetooth module.<br>3. Responsible for the design of the **pin wiring** for the car. |
| Yuhan Sheng | 1.Assisted in writing and debugging motor control code, and optimized motor operation logic.<br>2. Participated in **connecting and debugging the Bluetooth module**, and solved wiring issues.<br>3. Responsible for the aesthetic design of the car's appearance. |
| Xinyuan Bi | 1.Responsible for writing **the team's presentation PPT**, organizing project content.<br>2.Participated in modifying the MPU6500 module code. |
| Leilei Tao | 1.Responsible for making **the team's presentation PPT.**<br>2.Participated in debugging of code for the MPU6500 and Bluetooth modules. |
| Yuxuan Liu | 1.Responsible for **motor control logic and PID algorithm** optimization.<br>2.Responsible for debugging the car's motor control status and PWM duty cycle parameters. |
| Zexuan Dong | 1.Lead development of **radar module code**, including data collection, processing and parsing.<br>2.Responsible for **car appearance design** |

# Task division (Software)

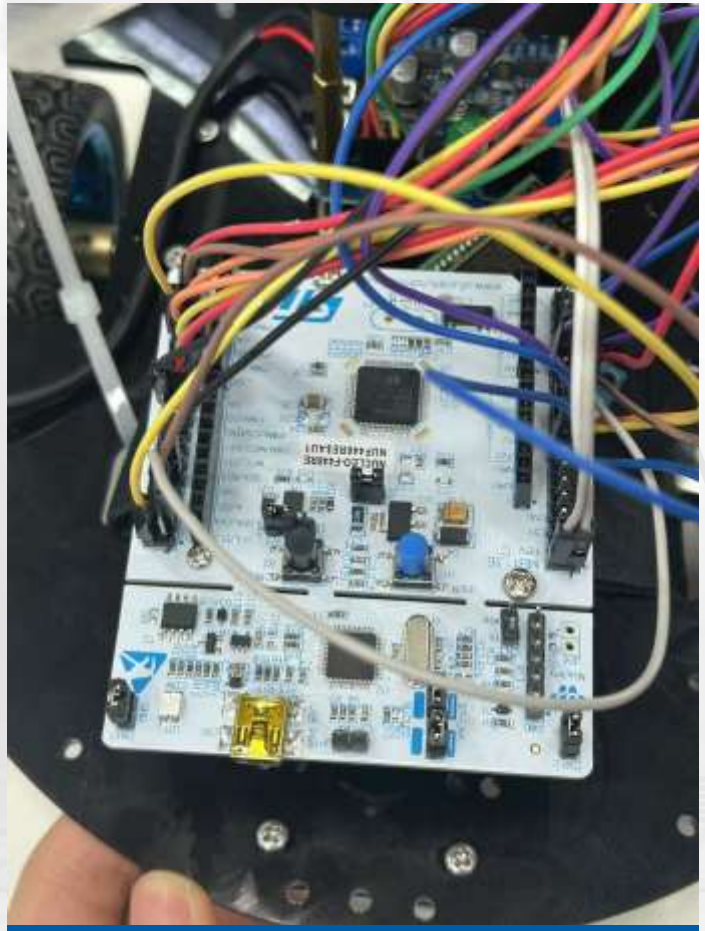
| Group members | Task division |
|---|---|
| **Mengzhe Shi (leader)** | 1.Designed and maintains the main control loop of the program.<br>2.Handled **serial communication between the PC and the robot**.<br>3.Managed switching between SIM and REAL operation modes. |
| **Ziyan Wei** | 1.**Processed LiDAR data** and performs basic data filtering.<br>2.Builded and updates the Occupancy Grid Map (OGM).<br>3.Assisted in improving localization accuracy through ICP fusion. |
| **Yuan Ma** | 1.Developed the **goal-seeking navigation logic**.<br>2.Implemented frontier-based exploration for autonomous mapping.<br>3.Tuned navigation parameters for smoother and safer robot movement. |
| **Yiran Zhao** | 1.Designed the user **interface and visualization** displays.<br>2.Handled simulation testing and system debugging. |

# Picture Record

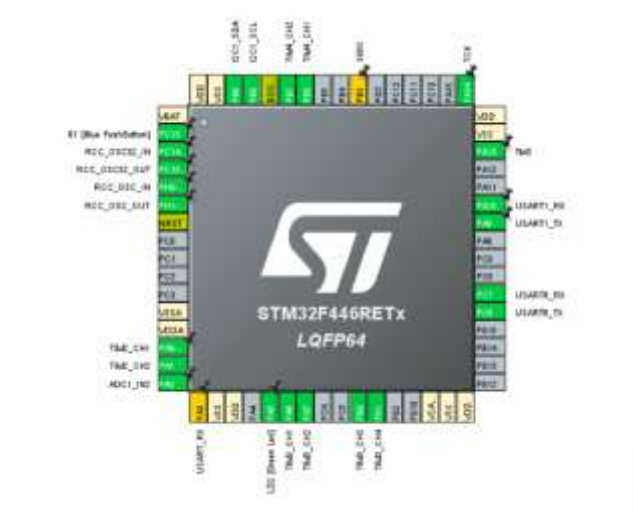
hardworking team members

# Module Introduction

# STM32F446RE



Physical image




Pin design in STM32CubeMX

| STM32 | 电机 | 雷达 | 蓝牙 | imu |
|---|---|---|---|---|
| PA6 | AIN1 | | | |
| PB0 | BIN1 | | | |
| PB6 | E2B | | | |
| PB7 | E2A | | | |
| PA7 | AIN2 | | | |
| PA1 | E1B | | | |
| PB1 | BIN2 | | | |
| PA0 | E1A | | | |
| PA10 | | | TXD | |
| PA9 | | | RXD | |
| 5V | 5V | | | |
| GND | GND | GND | GND | GND |
| PC7 | | TXD | | |
| PC6 | | RXD | | |
| | 5V | 5V | VCC | |
| PB8 | | | | SCL |
| PB9 | | | | SDA |
| | 3V3 | | | VCC |
| | GND | | | AD0 |

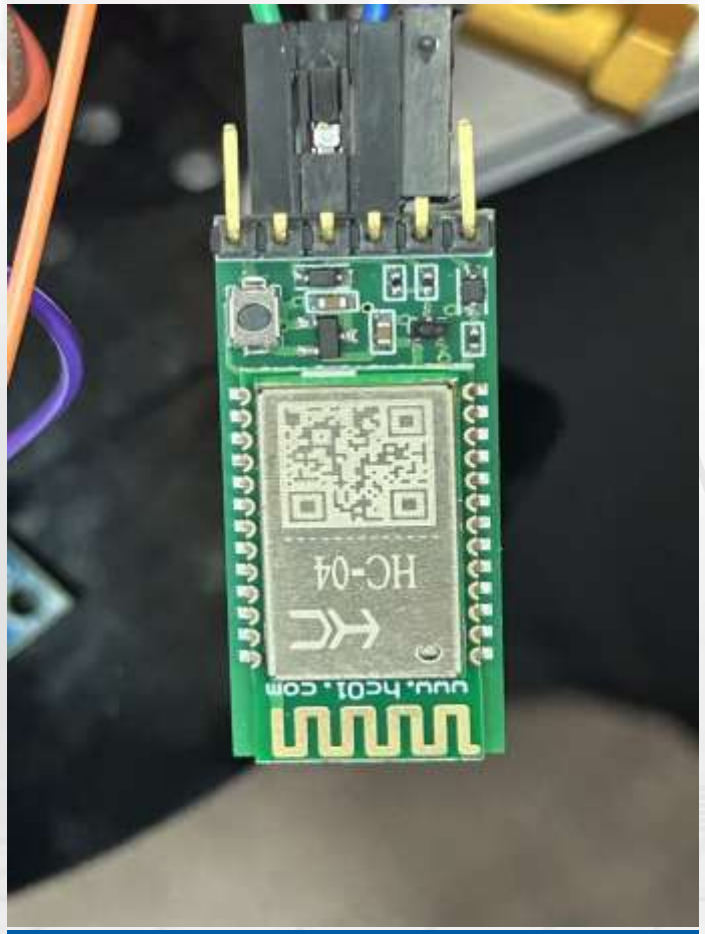
Connections between pins and various components (radar, IMU, encoder), and real-time processing of sensor data

Communicate with LiDAR+Bluetooth via UART, read MPU data, and capture encoder pulses

main function

Physical interface connection on the STM32F446RE board

# HC-04Bluetooth module




Physical image

## code example

```
// move directions
#define DIR_STOP 0                    // stop
#define DIR_FORWARD 1                 // forward
#define DIR_BACKWARD 2                // backward
#define DIR_LEFT 3                    // turn-left
#define DIR_RIGHT 4                   // turn-right
```

**Main functions:**
Send commands to the car via Bluetooth serial port to realize command transmission and data return between the robot and the computer.

**Transmitted content:**
connection status notifications, motor speed information, LiDAR scanning data, MPU6500 sensor data, and steering angle information.

# MPU6500 IMU

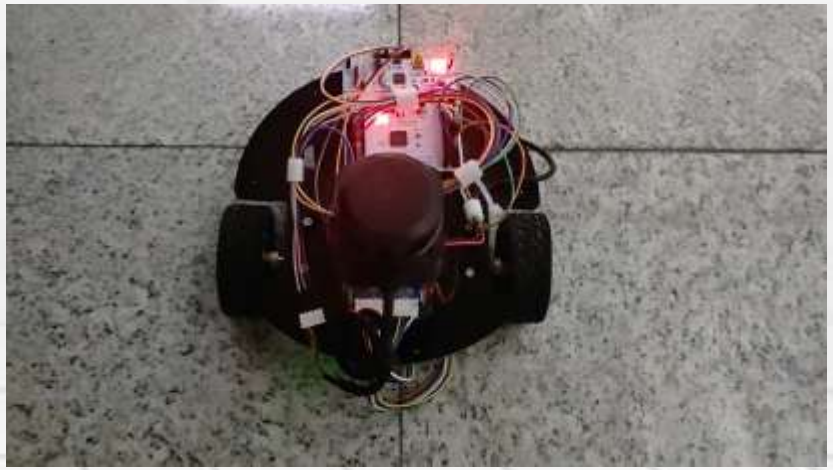
Physical image

**Main functions:**

The **angular velocity** of the three axes can be integrated to obtain the angle change, with zero drift error subtracted.

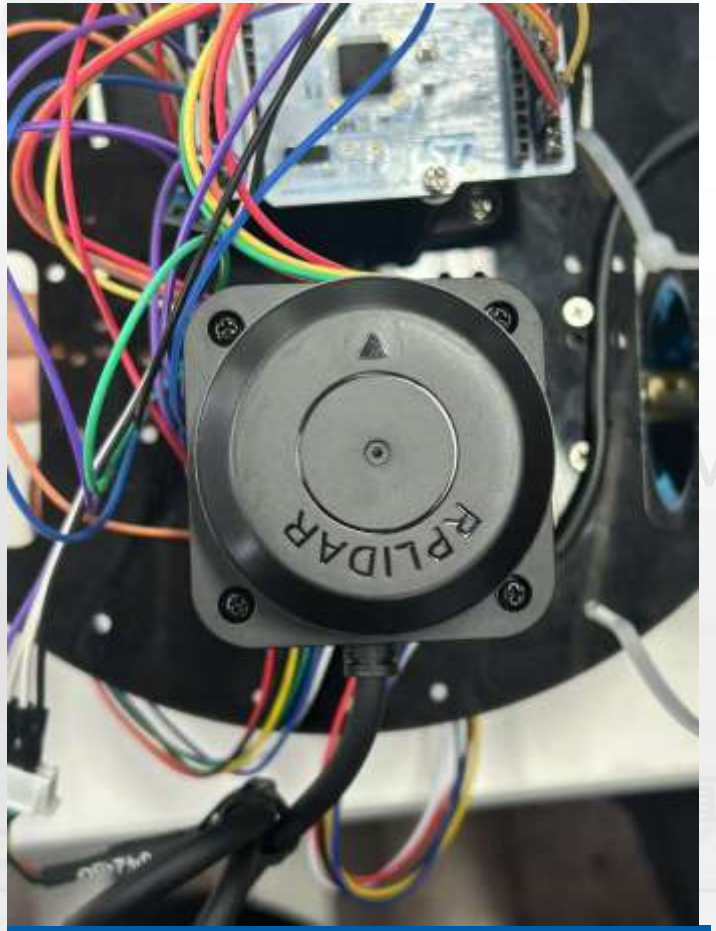
The output **three-axis acceleration** is used to determine the tilt state of the car through the gravity acceleration component, indirectly assisting in correcting the drift of the steering angle.

The **encoder** enables rotation through the differential drive of the two wheels, and the conversion formula from the rotated angle to the path length is used to assist in correcting the steering.



Our car can precisely control its rotation by 90 degrees!

# SLAMTEC RPLIDAR C1




Physical image

```
// ------------------------------------------------------------------------
// 激光雷达数据读取函数
// ------------------------------------------------------------------------
/**
 * @brief 激光雷达数据读取函数
 * @note 轮询方式读取激光雷达数据，避免复杂的DMA回调处理
 */
void ReadLidarData(void) {
  // 尝试从激光雷达串口读取数据
  if (HAL_UART_Receive(&huart6, lidar_rx_buffer, 1, 10) == HAL_OK) {
    uint8_t data = lidar_rx_buffer[0];

    // 寻找数据包头 (0xA5)
    if (lidar_packet_index == 0) {
      if (data == 0xA5) {
        lidar_data_packet[lidar_packet_index++] = data;
        lidar_last_rx_time = HAL_GetTick();
      }
    } else {
      // 收集数据包
      lidar_data_packet[lidar_packet_index++] = data;

      // 数据包完整时处理
      if (lidar_packet_index >= 5) {
        lidar_data_ready = 1;
        lidar_packet_index = 0;
      }
    }
  }

  // 数据包超时处理
  if (lidar_packet_index > 0 && (HAL_GetTick() - lidar_last_rx_time > 100)) {
    lidar_packet_index = 0;
  }
}

// ------------------------------------------------------------------------
// 处理激光雷达数据包
// ------------------------------------------------------------------------
/**
 * @brief 处理激光雷达数据包
 * @note 解析完整的激光雷达数据包并发送到蓝牙
 */
void ProcessLidarPacket(void) {
  if (!lidar_data_ready) return;

  // 检查数据包有效性
  if (lidar_data_packet[0] != 0xA5) {
    lidar_data_ready = 0;
    return;
  }
```

Code

The radar can achieve 360-degree data reading. However, sending a large amount of data to the computer via Bluetooth may cause packet loss and inaccurate mapping. Therefore, local filtering is performed: only the radar data within the range of four directions (±15 degrees) is transmitted.

# Goal-seeking navigation logic A*

```python
# ==================== 信息增益 A* 探索算法 ====================
def astar_info_gain(maze, start, goal):
    """信息增益 A*: 在启发式中加入未知区域奖励（未知邻居越多越有吸引力）"""
    def h(a,b):
        return abs(a[0]-b[0]) + abs(a[1]-b[1])

    def info_gain(a):
        x, y = a
        unknown = 0
        for dx,dy in [(0,1),(1,0),(0,-1),(-1,0)]:
            nx, ny = x+dx, y+dy
            if 0 <= nx < maze.size and 0 <= ny < maze.size:
                v = maze.cells[ny][nx]['visited']
                # 未访问邻格认为有信息增益
                if not v:
                    unknown += 1
        return unknown

    openh = [(h(start,goal), 0, start, [start])]
    closed = set()

    while openh:
        f,g,cur,path = heapq.heappop(openh)
        # 目标：到达第一个未访问格
        if not maze.cells[cur[1]][cur[0]]['visited']:
            return path
        if cur in closed:
            continue
        closed.add(cur)
        x,y = cur
        for dx,dy in [(0,1),(1,0),(0,-1),(-1,0)]:
            nx,ny = x+dx,y+dy
            if 0<=nx<maze.size and 0<=ny<maze.size and maze.can_move(x,y,nx,ny):
                gain = info_gain((nx,ny))
                cost = g + 1 - 0.3 * gain + random.uniform(-0.05,0.05)
                fscore = cost + h((nx,ny),goal)
                heapq.heappush(openh,(fscore, cost, (nx,ny), path+[(nx,ny)]))
    return None
```

Code

The system applies the A* algorithm for path planning on the occupancy grid map, finding safe and efficient routes based on cost and heuristic functions. ICP (Iterative Closest Point) is used to align consecutive LiDAR scans, correcting pose errors and reducing drift. SLAM integrates LiDAR data with pose estimation to achieve simultaneous localization and mapping, while a frontier-based exploration strategy automatically detects unexplored boundaries and selects new navigation targets for autonomous exploration.

# SLAM Mapping Logic

```python

# ==================== 地图类 ====================
class MazeMap:
    def __init__(self, from_json=None, hardware_mode=False):
        self.hardware_mode = hardware_mode
        if from_json and not hardware_mode:
            self.load_from_json(from_json)
        else:
            # 硬件模式：初始化未知地图（None），边界墙=True
            size = 4
            self.size = size
            self.cells = [[{'visited': False, 'walls': {'N': None, 'E': None, 'S': None, 'W': None}}
                           for _ in range(size)] for _ in range(size)]
            # 边界为实墙
            for y in range(size):
                for x in range(size):
                    if y == 0: self.cells[y][x]['walls']['S'] = True
                    if y == size-1: self.cells[y][x]['walls']['N'] = True
                    if x == 0: self.cells[y][x]['walls']['W'] = True
                    if x == size-1: self.cells[y][x]['walls']['E'] = True

    def load_from_json(self, filename, flip_x=False, flip_y=True):
        """仿真模式：读取 JSON 迷宫并翻转Y轴（保证 0,0/0,3 对齐）"""
        with open(filename, 'r', encoding='utf-8') as f:
            data = json.load(f)
        self.size = data['size']
        raw_rows = data['cells']
        rows = raw_rows[::-1] if flip_y else raw_rows
        processed = [row for row in rows]
        self.cells = [[{'visited': False, 'walls': cell['walls']} for cell in row]
                      for row in processed]

    def wall_val(self, x, y, d):
        """检测墙壁：True=有墙，False=通路，None=未知"""
        return self.cells[y][x]['walls'][d]

    def can_move(self, x, y, nx, ny):
        """是否可通行：硬件模式下未探索(None)视为不可通行，仿真模式按JSON墙判断"""
        if not (0 <= nx < self.size and 0 <= ny < self.size):
```

Code

The SLAM process begins with collecting the robot’s pose and LiDAR scan data. ICP aligns the current scan with the previous one or a local map to refine the pose estimation. Using an inverse sensor model, the system updates the occupancy grid map by marking free and occupied spaces according to LiDAR beam reflections. As the robot moves, the map is continuously expanded, and the navigation module uses A* or frontier-based methods to plan feasible paths and enable exploration in unknown environments.

# LiDAR Signal Processing Logic

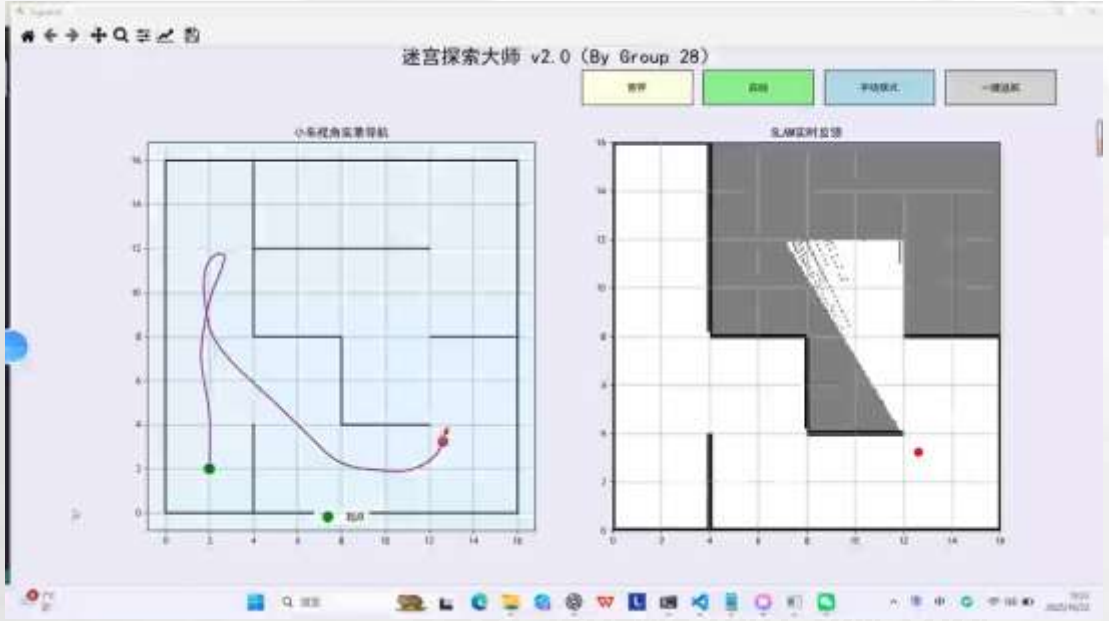
```python
import time

# ============== 运行模式开关 ==============
USE_HARDWARE = True   # True: 串口+真实小车; False: 纯软件仿真

if USE_HARDWARE:
    # 需要你已创建 serial_bridge.py 并能收发
    from serial_bridge import send_command, read_sensor
else:
    # 仿真占位：不发命令、用虚拟OBS
    def send_command(cmd):
        print(f"[SIM] 发送命令: {cmd}")
    def read_sensor():
        return None  # 仿真模式下我们从地图直接生成OBS，不依赖串口

```

**Code**

The LiDAR processing module receives and verifies scan data via serial communication, extracting angles and distances from each frame. The data is filtered to remove noise and invalid points, then converted from polar coordinates to the robot and global coordinate systems. The processed point cloud is combined with pose data and fed into the SLAM module to update the map and correct localization in real time, ensuring stable and reliable navigation data.

# Completion

# Completion Video

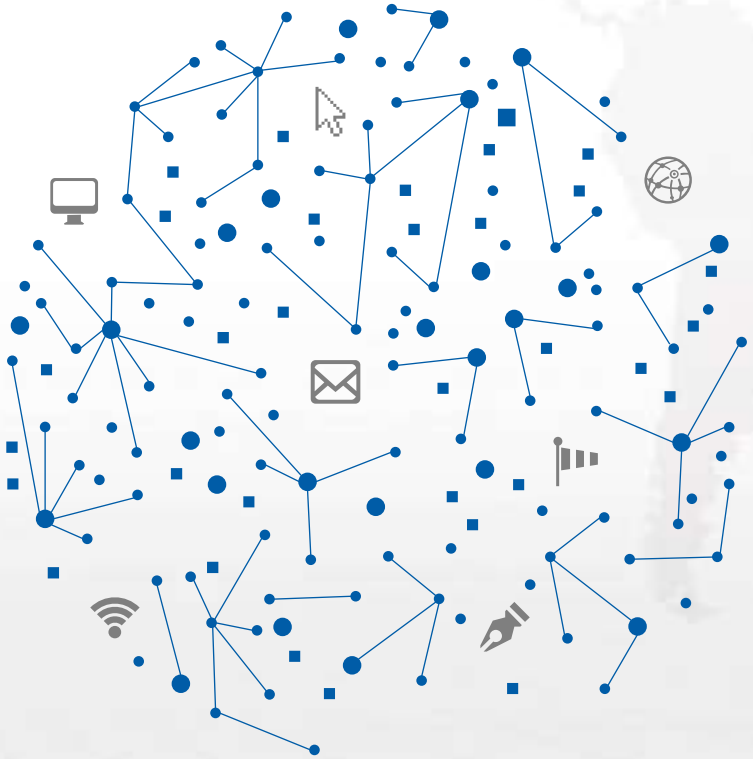


# What work have we completed?

- **Basic Level Completion – Fundamental Integration and Autonomous Operation**
  - ✓ The robot successfully completes low-level control using PID for motion toward given target positions.
  - ✓ Firmware reliably computes odometry and current position (x,y,θ) and transmits 2D LiDAR and position data to the host PC via Bluetooth.
  - ✓ The system responds to navigation waypoint commands from the host PC.
  - ✓ Host PC software visualises robot movement and LIDAR data in real-time using PyRoboViz.
  - ✓ A simple autonomous navigation behaviour with dynamic obstacle avoidance is demonstrated.
  - ✓ SLAM or mapping is not included at this level.

- **Intermediate Level Completion – Autonomous Exploration and Mapping**
  - ✓ All Basic Level functionalities are complete and fully integrated.
  - ✓ The robot autonomously explores unknown environments using improved navigation logic (e.g., frontier-based exploration).
  - ✓ A dynamic SLAM map construction of the environment using BleezySLAM.
  - ✓ The host PC visualises both the evolving SLAM constructed map and real-time robot trajectory using PyRoboViz.
  - ✓ The software detects and marks the Exit Point within the map as part of its autonomous behaviour.

- **Advanced Level Completion – Return Navigation and Mission Completion**
  - ✓ All Intermediate Level functionalities are successfully implemented and stable.
  - ✓ The robot plans and navigates a path from the Exit Point back to the Starting Position using a path planning algorithm (e.g., A*, DWA).
  - The system maintains localisation, avoids obstacles, and executes the return path autonomously.
  - Advanced features are added, such as a user interface, real-time parameter tuning, or extended capabilities (e.g., adaptive behaviour, performance analysis).

**We have already implemented the intermediate-level required functions!**

# Thank you for watching!